

`int arr[5]` →

--	--	--	--	--

↓
integer array with 5 elements

`int arr[N]` →

--	--	--	--	--	--	--	--	--	--

↓
integer array with N elements

// print an entire array

```
for (i=0; i<N; i++) {  
    print(arr[i])  
}
```

⇒ TC: $O(N)$

accessing element from i^{th} index ⇒ $\text{arr}[i] \rightarrow \underline{O(1)}$

`int arr[5] ⇒`

--	--	--	--	--

0 1 2 3 4

4th element
3rd index

get the 4th element ⇒ `arr[3]`

Q. 1. Given N array element, count no. of element having at least 1 element greater than itself [no lib. func.]

arr[7] $\Rightarrow \{ \underset{\checkmark}{-3} \underset{\checkmark}{-2} \underset{\checkmark}{6} \underset{\times}{8} \underset{\checkmark}{4} \underset{\times}{8} \underset{\checkmark}{5} \} \xrightarrow{\text{largest} | \text{count}} 8 \quad 2$

Qp = 5.

arr[8] $\Rightarrow \{ \underset{\checkmark}{2} \underset{\checkmark}{3} \underset{\times}{10} \underset{\checkmark}{7} \underset{\checkmark}{3} \underset{\checkmark}{2} \underset{\times}{10} \underset{\checkmark}{8} \} \Rightarrow 10 \mid 2$

Qp = 6.

arr[10] $\Rightarrow \{ \underset{\checkmark}{2} \underset{\checkmark}{5} \underset{\checkmark}{1} \underset{\checkmark}{4} \underset{\checkmark}{8} \underset{\checkmark}{0} \underset{\checkmark}{8} \underset{\times}{13} \underset{\checkmark}{8} \} \Rightarrow 13 \mid 1$

Qp = 9.

Bruteforce

```
for (i=0; i<N; i++) {
```

```
    for (j=0; j<N; j++) {
```

```
        if (any bigger element found)
```

```
            c++; break;
```

```
    }
```

```
}
```

```
return c;
```

$\therefore \Rightarrow \underline{\underline{O(N^2)}}$

Observation

- I) every element except the largest element will have any element bigger than itself
- II) $ans = \text{all element except largest elements}$
 $\downarrow$
 $ans = N - (\text{count of largest of element})$

pseudo

- I) iterate to get max^m
- II) iterate the get count of max^m
- III) $ans = N - \text{count of } max^m$

$\nwarrow$ $maxV = 0$ $\nearrow$ not correct, if all element -ve, it won't work

valid values

```
maxV = INT_MIN
maxV = arr[0]
for (i = 0; i < N; i++) {
    if (maxV < arr[i]) {
        maxV = arr[i]
    }
}

c = 0
for (i = 0; i < N; i++) {
    if (maxV == arr[i])
        c++
}
```

return arr.length - c;

ex $\Rightarrow$ arr[6] $\Rightarrow$ [2 2 2 2 2 2]

Qp = 0.

max^m = 2

count = 6

ans = 2 - count
= 6 - 6 = 0

TC $\Rightarrow O(2N) \approx O(N)$

SC $\Rightarrow O(1)$.

How $\Rightarrow$ change this code, and solve in a single loop.

Q2 Given N array elements, check if there exists a pair (i, j) such that $arr[i] + arr[j] = k$, & $i \neq j$. Note: i & j are indices, k is given sum.

arr $\Rightarrow$

3	-2	1	4	3	6	8
0	1	2	3	4	5	6

k = 10.

Qp = true $\Rightarrow$ arr[3] + arr[5] = 10

arr $\Rightarrow$ 2 4 -3 7
 0 1 2 3

$k = 5$

Qp $\Rightarrow$ false.

arr $\Rightarrow$ 2 4 -3 7
 0 1 2 3

$k = 8$

Qp $\Rightarrow$ false

$(arr[1] + arr[1] == 8)$
 XXX not-allowed.

solⁿ

check for all pairs.

$N = 5 \Rightarrow \{ \underline{0} \quad \underline{1} \quad \underline{2} \quad \underline{3} \quad \underline{4} \}$

pairs $\Rightarrow$

(0,0)	(1,0)	(2,0)	(3,0)	(4,0)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)
(0,4)	(1,4)	(2,4)	(3,4)	(4,4)

Brute force

```
for (i = 0; i < N; i++) {  
    for (j = 0; j < N; j++) {  
        if (i == j) continue;  
        if (arr[i] + arr[j] == 2 * k)  
            return true  
    }  
}  
return false
```

iterations $\Rightarrow N * N \Rightarrow N^2$

TC $\Rightarrow O(N^2)$

SC $\Rightarrow O(1)$

Optimised

```
for (i = 0; i < N; i++) {  
    for (j = i + 1; j < N; j++) {  
        if (arr[i] + arr[j] == 2 * k)  
            return true  
    }  
}  
return false
```

$$N-1 \times 1$$

$$N-1$$

$$N-1 - 2 + 1$$

$$= N-2$$

i	j	iterations
0	$[1 - (N-1)]$	$N-1$
1	$[2 - (N-1)]$	$N-2$
2	$[3 - (N-1)]$	$N-3$
⋮		⋮
$N-1$	$[N - (N-1)]$	<u><u>0</u></u>

$$N-1 + N-2 + N-3 + \dots + 3 + 2 + 1 + 0$$

$$\Rightarrow 1 + 2 + 3 + \dots + N-1$$

$$\Rightarrow \frac{(N-1) \times (N-1 + 1)}{2} \Rightarrow \frac{N(N-1)}{2} \text{ iterations}$$

$$B.F \Rightarrow N^2 \text{ iterations} \Rightarrow TC \Rightarrow O(N^2)$$

$$\text{Optimised} \Rightarrow \frac{N(N-1)}{2} \text{ iterations} \Rightarrow TC \Rightarrow O(N^2)$$

building

Java revisited

How to do it in Java

Scalar topics

Q3, Given an array, reverse entire array. | Sc: $O(1)$.

arr $\Rightarrow$

-1	4	7	6	-2	7	8	10
0	1	2	3	4	5	6	7

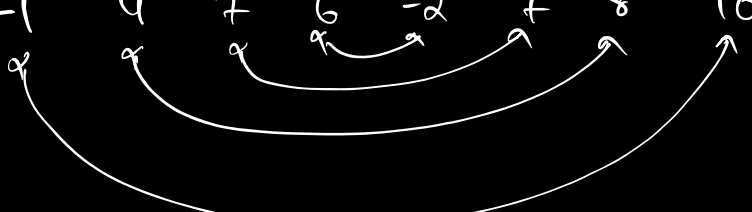
rev. arr $\Rightarrow$

10	8	7	-2	6	7	4	-1
0	1	2	3	4	5	6	7

Sc $\Rightarrow O(1) \rightarrow$ no extra array
given array itself should change (get reversed)

arr $\Rightarrow$

0	1	2	3	4	5	6	7
-1	4	7	6	-2	7	8	10



Result $\Rightarrow$

10	8	7	-2	6	7	4	-1
----	---	---	----	---	---	---	----

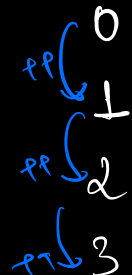
arr $\Rightarrow$

0	1	2	3	4	5	6
-1	6	3	2	8	9	10



i

0
1
2
3



j

6
5
4
3

$\rightarrow$ swap.
 $\rightarrow$ swap.
 $\rightarrow$ swap.
 $\rightarrow$ swap.

pseudo

reverse(arr, N) {

 i = 0

 j = N-1

 while(i <= j) {

 swap(arr[i], arr[j])

 i++

 j--

 }

 return arr

}

TC $\Rightarrow O(N)$

SC $\Rightarrow O(1)$

Q4. Given N array elements, & $[Si, Ei]$, reverse array from $[Si - Ei]$, note $\Rightarrow Si \leq Ei$.

arr $\Rightarrow$ 0 1 2 3 4 5 6 7 8
 -3 4 2 8 7 9 6 2 10

$Si = 3$

$Ei = 7$

rev. arr $\Rightarrow$ -3 4 2 2 6 9 7 8 10.

pseudo

reversePart(arr, si, ei) {

i = si, j = ei

while (i <= j) {

swap(arr[i], arr[j])

i++

j--

}

}

TC $\Rightarrow O(N)$

SC $\Rightarrow O(1)$

Q5. Given N array elements, rotate array from last to first by k times.

arr[7] $\Rightarrow$

k = 1

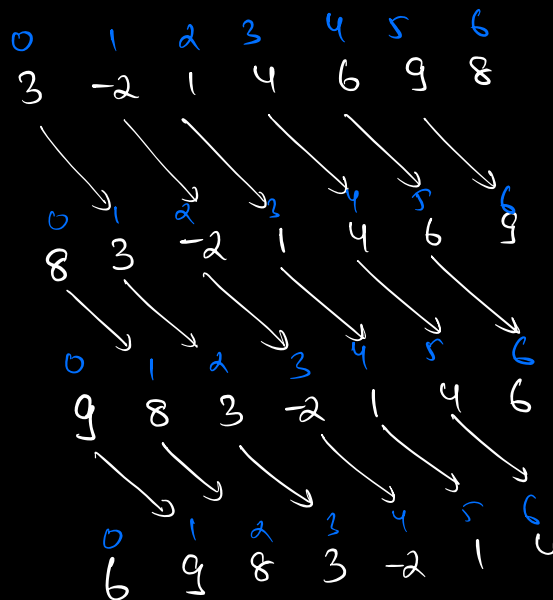
rotated array $\Rightarrow$

k = 2

rotated array $\Rightarrow$

k = 3

rotated array $\Rightarrow$



$$\text{arr}[9] = 4 \ 1 \ 6 \ 9 \ 2 \ 14 \ 7 \ 8 \ 3$$

$k=4$

$$\text{rot arr} = 14 \ 7 \ 8 \ 3 \ 4 \ 1 \ 6 \ 9 \ 2$$

$$\text{arr}[10] = -2 \ 3 \ 1 \ 4 \ 6 \ 2 \ 8 \ 7 \ 9 \ 3$$

$k=3$

$$\text{rot arr} = 7 \ 9 \ 3 \ -2 \ 3 \ 1 \ 4 \ 6 \ 2 \ 8$$

last 3 element of original $\Rightarrow$ first 3 elements of rotated array

first 7 elements of original $\Rightarrow$ last 7 element of rotated array

$$\text{arr}[13] \Rightarrow a_0 \ a_1 \ a_2 \ a_3 \ a_4 \ a_5 \ a_6 \ a_7 \ a_8 \ a_9 \ a_{10} \ a_{11} \ a_{12}$$

$$k=5$$

$$\text{rot arr} \Rightarrow a_8 \ a_9 \ a_{10} \ a_{11} \ a_{12} \ a_0 \ a_1 \ a_2 \ a_3 \ a_4 \ a_5 \ a_6 \ a_7$$

$$\text{reverse} \Rightarrow a_{12} \ a_{11} \ a_{10} \ a_9 \ a_8 \ a_7 \ a_6 \ a_5 \ a_4 \ a_3 \ a_2 \ a_1 \ a_0$$

reverse first k element

reverse last (N-k) element

$$a_8 \ a_9 \ a_{10} \ a_{11} \ a_{12} \ a_0 \ a_1 \ a_2 \ a_3 \ a_4 \ a_5 \ a_6 \ a_7$$

pseudo

$$\Rightarrow k \geq k \% N$$

i) reverse the entire array $\rightarrow \text{reversePart}(\text{arr}, 0, N-1)$

ii) reverse the first k element $\rightarrow \text{reversePart}(\text{arr}, 0, k-1)$

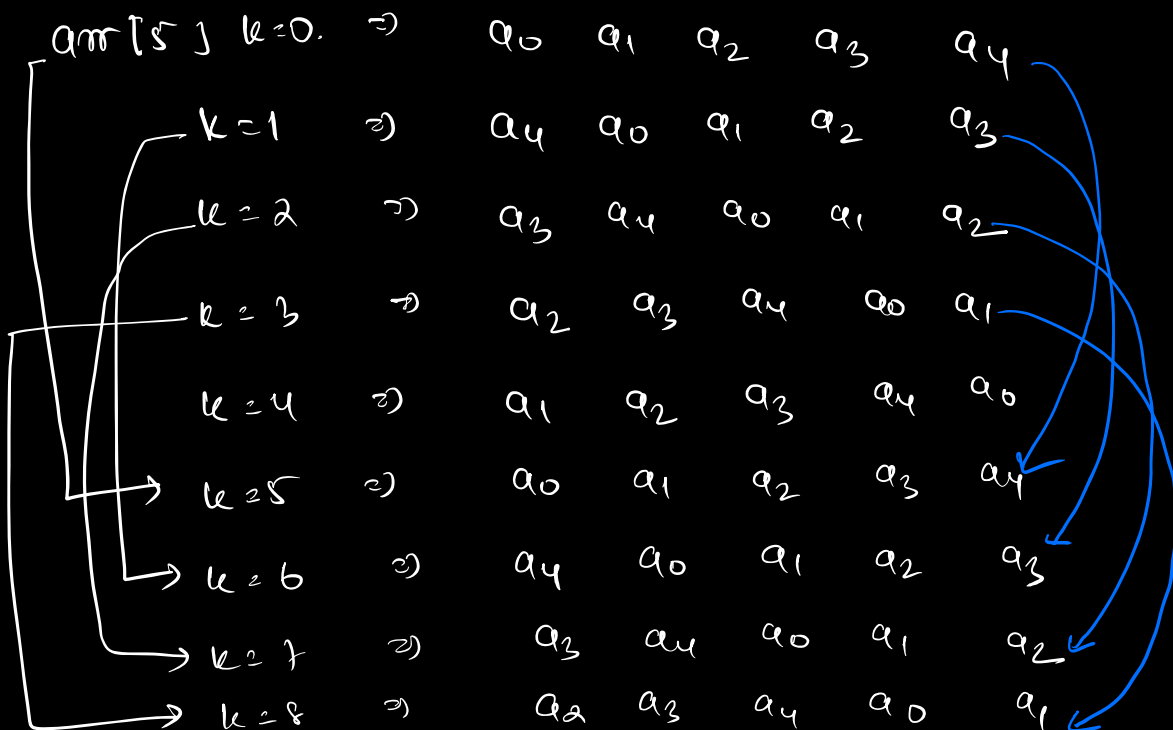
iii) reverse the last $N-k$ element $\rightarrow \text{reversePart}(\text{arr}, k, N-1)$

$$TC \Rightarrow O(N + k + N - k)$$

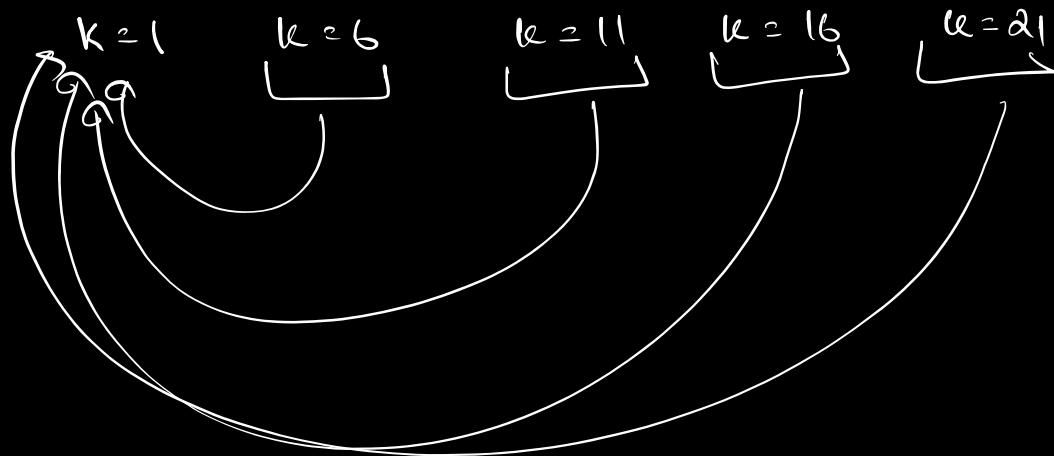
$$= O(2N) \approx O(N)$$

$$SC \Rightarrow O(1)$$

$$k > N \Rightarrow \text{reversePart}(\text{arr}, 0, \underbrace{(k-1)}_{\geq N-1})$$



$N = 5$



$k = k \% N$
↓
rotate for k

Optional

How $\Rightarrow$ Rotate k time ($L \rightarrow R$)

Static length array $\rightarrow$ arr[5] $\rightarrow$ fixed arrays.

dynamic arrays $\rightarrow$ arrays with dynamic length

C++ $\rightarrow$ vector

Java $\rightarrow$ arraylist

Python $\rightarrow$ list

C# $\rightarrow$ arraylist / list

JS $\rightarrow$ array

C $\rightarrow$ change to C++ (Java).

list < int > l

↓ ↓

datatype name

l.size() → 0

l.insert(1)

l.insert(2)

l.insert(3)

l.size() → 3

0	1	2
1	2	3

print(l.get(1)) → 2

TC →

l.get()	}	<u>OC(1)</u>
l.insert()		
l.size()		

// print all elements of a list

for(i=0; i < l.size(); i++) {

 print(l.get(i))

}
